BLS Producer — Full Engineering Documentation

> **Version**: 1.0 · **Last Updated**: 2025-07
> **Product**: BLS Producer → Rich Media Creator → Web Design Suite
> **Audience**: Engineers building, maintaining, and scaling this application

---

Table of Contents

1. Product Overview
2. Technology Stack
3. Project Structure
4. Getting Started
5. Data Model
6. State Management (Zustand)
7. Persistence & Storage
8. Layout & UI Architecture
9. Canvas & Rendering Engine
10. Drag, Resize & Snapping
11. Rich Text Editing (TipTap)
12. Animation System
13. Export Engine
14. Sanity Cloud Integration
15. Internationalisation (i18n)
16. Coding Conventions & CI
17. Feature Priority List
18. Architecture Evolution Plan
19. Roadmap — Phase-by-Phase
20. Cross-Cutting Concerns
21. Open Architecture Decisions

---

1. Product Overview

BLS Producer is a **browser-based interactive ad and rich-media creator**. Users design multi-frame animated creatives (banner ads, quizzes, interactive stories) via a visual editor and export them as **self-contained HTML files** with no external runtime dependencies.

Core Capabilities (Current — v1.x)

Capability	Description
Multi-frame editor	Canvas with absolute-positioned objects per frame
Object types	Text, Image, Answer Group, Shape, Divider, Path (SVG)
Rich text	TipTap WYSIWYG editing with inline font/colour/weight
Animations	Preset + custom keyframe animations (enter / exit / loop)
Interactions	Hover + click effects per object
CSS Filters	Brightness, contrast, blur, saturation, opacity, etc.
Background	Colour, gradient, image with Ken Burns / zoom animations
Export	Self-contained HTML zip (Google Fonts + embedded assets)
Cloud save	Save/load projects to Sanity CMS
Assets	Image upload, custom font upload, Google Fonts integration
Localisation	Duplicate frames per locale, translation map

Evolution Path

```
BLS Producer (v1.x)          — current
  ↓
Rich Media Creator (v2.x)    — vector engine + effects + interactions
  ↓
Timeline Studio (v3.x)       — keyframe timeline, game templates
  ↓
Responsive Designer (v4.x)   — auto-layout, breakpoints, device preview
  ↓
Web Design Suite (v5.x)      — multi-page sites, components, design tokens
```

---

2. Technology Stack

Runtime Dependencies

Package	Version	Purpose
Next.js	16.1.6	App Router + Turbopack dev server
React	19.2.3	UI rendering
Mantine	8.3.x	Component library (panels, inputs, tabs, accordions)
Zustand	5.0.x	Global state management
TipTap	3.20.x	Rich text editor (extensions: colour, font-family, highlight, alignment, underline, text-style)
dnd-kit	6.3.x / 10.x	Drag-and-drop (layers reorder)
Sanity	5.12.x	CMS Studio + image URL builder + GROQ client
fflate	0.8.2	Zip compression for export
idb-keyval	6.2.2	IndexedDB get/set for project persistence
styled-components	6.3.x	Used by Sanity Studio internally
Tabler Icons	3.38.x	Icon set for UI
dayjs	1.11.x	Date formatting

Dev Tooling

Tool	Purpose
TypeScript 5 (strict)	Type safety; `--noEmit` check
ESLint 9	Linting; `--max-warnings=0` enforcement
Husky + lint-staged	Pre-commit hooks: ESLint auto-fix on staged files
Commitlint	Conventional Commits format enforcement
Tailwind CSS 4	PostCSS integration (minimal use; Mantine is primary)

Build Pipeline

```
npm run verify:strict
  → tsc --noEmit         (typecheck)
  → eslint --max-warnings=0  (lint)
  → next build           (production build)
```

Configuration Files

File	Purpose
`next.config.ts`	Allows `cdn.sanity.io` for remote images
`tsconfig.json`	Strict mode, `@src/*` path alias → project root
`eslint.config.mjs`	ESLint flat config
`postcss.config.mjs`	Tailwind PostCSS plugin
`sanity.config.ts`	Sanity Studio configuration
`sanity.cli.ts`	Sanity CLI config

---

3. Project Structure

```
bls-producer-next/
├── app/                        # Next.js App Router
│   ├── layout.tsx              # Root layout (MantineProvider, fonts)
│   ├── page.tsx                # Main editor page
│   ├── globals.css             # Global styles
│   ├── api/
│   │   ├── cloud/
│   │   │   ├── projects/       # GET: list Sanity projects
│   │   │   ├── save/           # POST: save project to Sanity
│   │   │   └── upload-images/  # POST: upload images to Sanity
│   │   └── proxy-image/
│   │       └── route.ts        # GET: CORS proxy for image export inlining
│   └── studio/
│       └── [[...tool]]/
│           └── page.tsx        # Sanity Studio embedded route
│
├── components/
│   ├── canvas/                 # Canvas rendering layer
│   │   ├── Board/              # Scroll+zoom container, drop target
│   │   ├── BoardContextMenu/   # Right-click menu on board
│   │   ├── ContextMenu/        # Shared context menu component
│   │   ├── FrameCard/          # Single frame slot (bg + objects)
│   │   ├── FrameObject/        # Object renderer + interaction handler
│   │   │   ├── FrameObject.tsx         # Main component (selection, drag, resize handles)
│   │   │   ├── FrameObjectRenderers.tsx # Type-specific renderers (Image, Shape, Divider, Path, AnswerGroup)
│   │   │   ├── frameObjectUtils.ts     # Shared utilities
│   │   │   └── InlineTextEditor.tsx    # TipTap editor (mounted on double-click)
│   │   └── RuledCanvas/        # Ruler overlay (px markers, cursor lines)
│   │
│   ├── layout/                 # Application shell
│   │   ├── AppHeader/          # Top toolbar (context-sensitive per selection)
│   │   ├── BlsProducer/        # Root layout orchestrator
│   │   ├── BottomPanel/        # Frame timeline strip
│   │   ├── RightPanel/         # 5 detachable panels (Layers, Props, Anim, Filters, Frame)
│   │   └── Sidebar/            # Left sidebar (frame thumbnails, global settings)
│   │
│   ├── modals/                 # Dialog/modal components
│   │   ├── AdPreviewModal/     # Full preview playback
│   │   ├── AiPromptModal/      # AI prompt dialog
│   │   ├── CloudProjectsModal/ # Load from cloud
│   │   ├── ErrorBoundary/      # Error catch boundary
│   │   ├── KeyboardShortcutsModal/ # Keyboard shortcuts reference
│   │   ├── SaveToCloudModal/   # Save to Sanity cloud
│   │   └── TemplateGallery/    # Template browser
│   │
│   └── sidebar/                # Right-panel sub-sections
│       ├── AnimPanel/          # Per-object animation editor
│       ├── AnswerGroupFields/  # Answer group properties
│       ├── BgFillSection/      # Background fill controls
│       ├── BgImageSection/     # Background image controls
│       ├── ClickEffectPanel/   # Click interaction editor
│       ├── DividerObjectFields/ # Divider properties
│       ├── EasingPicker/       # Easing curve selector
│       ├── FiltersBlendPanel/  # CSS filters + blend mode
│       ├── FontFamilySelect/   # Font picker (Google + custom)
│       ├── FrameAnimPanel/     # Frame-level animation settings
│       ├── FrameListSection/   # Frame list in sidebar
│       ├── FrameSizeSection/   # Frame W/H controls
│       ├── HoverEffectPanel/   # Hover interaction editor
│       ├── ImageObjectFields/  # Image properties
│       ├── KeyframeEditor/     # Custom keyframe editor
│       ├── LoopAnimPanel/      # Loop animation picker
│       ├── ObjectEditorSection/ # Properties panel orchestrator
│       ├── ShapeObjectFields/  # Shape properties
│       └── TextObjectFields/   # Text properties (padding, typography)
│
├── context/
│   └── DragContext.ts          # React context for drag/resize handlers
│
├── hooks/
│   ├── dragHandlers.ts         # Pointer event handlers for drag
│   ├── dragTypes.ts            # Drag type definitions
│   ├── useDrag.ts              # Main drag + resize hook
│   ├── useExport.ts            # Export orchestration hook
│   ├── useRuler.ts             # Ruler/cursor position hook
```

```
│   └── useSanityCloud.ts       # Sanity cloud operations hook
│
├── lib/
│   ├── align.ts                # Alignment utilities (left, center, right, distribute)
│   ├── animCompiler.ts         # CSS @keyframes compiler
│   ├── animPresets.ts          # Named animation presets
│   ├── exportEngine.ts         # HTML export engine (~870 lines)
│   ├── fonts.ts                # System fonts set + helpers
│   ├── googleFontsList.ts      # Google Fonts catalogue
│   ├── idbStorage.ts           # IndexedDB persistence layer
│   ├── insertHelpers.ts        # Object insertion utilities
│   ├── presetLibrary.ts        # Shape/object preset library
│   ├── snapGuides.ts           # Snap-to-edge/centre guide computation
│   ├── templates.ts            # Project template definitions
│   ├── theme.ts                # Mantine theme config
│   ├── types.ts                # Core data model types (~250 lines)
│   ├── sanity/
│   │   ├── client.ts           # Sanity query client
│   │   ├── imageUtils.ts       # Image URL helpers
│   │   ├── queries.ts          # GROQ queries
│   │   └── schema/             # Sanity schema definitions
│   └── templates/
│       ├── _shared.ts          # Shared template utilities
│       └── quiz/               # Quiz template definitions
│
├── public/                     # Static assets
│
├── sanity/                     # Sanity Studio config (for /studio route)
│   ├── env.ts                  # Environment variables
│   ├── structure.ts            # Studio document structure
│   ├── lib/
│   │   ├── client.ts           # Studio Sanity client
│   │   ├── image.ts            # Image builder
│   │   └── live.ts             # Live preview
│   └── schemaTypes/
│       └── index.ts            # Schema type exports
│
├── store/
│   ├── quizStore.ts            # Zustand store (composed from 6 slices)
│   ├── types.ts                # Store types, actions, helpers (~234 lines)
│   └── slices/
│       ├── clipboardSlice.ts   # Copy/paste object
│       ├── cloudSlice.ts       # Cloud project metadata
│       ├── frameSlice.ts       # Frame CRUD, reorder, sizing
│       ├── historySlice.ts     # Undo/redo snapshot stack
│       ├── objectSlice.ts      # Object CRUD, z-order, lock/hide
│       └── selectionSlice.ts   # Selection state, UI toggles
│
├── documentation/
│   └── features-list-by-priority.txt  # Feature priority list
│
├── package.json
├── tsconfig.json
└── next.config.ts
```

---

4. Getting Started

Prerequisites

- Node.js ≥ 20
- npm ≥ 10

Installation

```
git clone <repo-url>
cd bls-producer-next
npm install
```

Environment Variables

Create `.env.local` with Sanity credentials (see `sanity/env.ts` for required keys):

```
NEXT_PUBLIC_SANITY_PROJECT_ID=your_project_id
NEXT_PUBLIC_SANITY_DATASET=production
SANITY_API_TOKEN=your_write_token
```

Development

```
npm run dev          # Start dev server (Turbopack, port 3000)
```

Quality Checks

```
npm run typecheck    # TypeScript check (tsc --noEmit)
npm run lint         # ESLint
npm run lint:strict  # ESLint with --max-warnings=0
npm run verify       # typecheck + build
npm run verify:strict # typecheck + lint:strict + build
```

Routes

Path	Description
`/`	Main editor application
`/studio`	Sanity Studio (CMS admin)
`/api/cloud/projects`	REST: list cloud projects
`/api/cloud/save`	REST: save project
`/api/cloud/upload-images`	REST: upload images
`/api/proxy-image?url=...`	REST: CORS image proxy

---

5. Data Model

All types are defined in `lib/types.ts` and `store/types.ts`.

Top-Level: `QuizData`

```
interface QuizData {
  bg: string | null; // global background colour
  frames: Frame[]; // ordered frame list
  customCss?: string; // user-injected CSS
  translations?: Record<string, Record<string, string>>; // locale → objId → text
}
```

Frame

```
interface Frame {
  id: string;
  src: string | null; // unused legacy field
  objects: FrameObject[]; // objects on this frame
  w: number; // frame width (px)
  h: number; // frame height (px)
  isDefault?: boolean;
  isEndFrame: boolean; // marks the final "end" frame
  animEnter: AnimConfig; // frame enter transition
  animExit: AnimConfig; // frame exit transition
  answerStagger: number; // ms stagger between answer buttons
  enterStagger?: number; // ms gap between animOrder waves (enter)
  exitStagger?: number; // ms gap between animOrder waves (exit, reversed)
  bgColor?: string;
  bgGradient?: { angle: number; stops: [string, string] } | null;
  bgImage?: string | null;
  bgImageAnim?: BgImageAnim | null;
  bgImageSize?: "cover" | "contain" | "auto" | string;
  bgImagePosX?: number; // bg position X offset (%)
  bgImagePosY?: number; // bg position Y offset (%)
  locale?: string; // language identifier for i18n frames
}
```

FrameObject (Union Type)

```
type FrameObject =
  | TextObject
  | ImageObject
  | AnswerGroupObject
  | ShapeObject
  | DividerObject
  | PathObject;
```

BaseObject (shared fields)

```typescript
interface BaseObject {
  id: string;
  label: string; // display name in layers panel
  x: number; // absolute X position (px)
  y: number; // absolute Y position (px)
  role?: string; // semantic role (e.g. "heading", "cta")
  animIn?: AnimConfig; // enter animation preset
  animOut?: AnimConfig; // exit animation preset
  animLoop?: LoopAnimConfig; // loop animation preset
  hoverEffect?: HoverEffect;
  clickEffect?: ClickEffect;
  customAnimIn?: CustomAnim; // custom keyframe enter
  customAnimOut?: CustomAnim; // custom keyframe exit
  customAnimLoop?: CustomAnim; // custom keyframe loop
  zIndex?: number;
  opacity?: number; // 0–100
  rotation?: number; // degrees
  hidden?: boolean; // layers panel visibility toggle
  locked?: boolean; // layers panel lock toggle
  animOrder?: number; // stagger wave index
  blendMode?: string; // CSS mix-blend-mode
  cssFilter?: CSSFilterConfig;
}
```

TextObject

```typescript
interface TextObject extends BaseObject {
  type: "text";
  text: string; // plain text content
  richText?: string; // TipTap HTML (overrides text when present)
  size: number; // font size (px)
  color: string; // text colour
  w?: number; // explicit container width; undefined = shrink-to-content
  bgColor?: string; // background fill colour
  bgEnabled?: boolean; // enable/disable background fill
  radius?: number; // border-radius (px)
  paddingX?: number; // horizontal padding (px)
  paddingY?: number; // vertical padding (px)
  paddingTop?: number; // individual paddings (override paddingX/Y)
  paddingRight?: number;
  paddingBottom?: number;
  paddingLeft?: number;
  fontFamily?: string;
  fontWeight?: string; // '400' | '700' | '900' …
  letterSpacing?: number; // px
  lineHeight?: number; // multiplier (e.g. 1.2)
  textAlign?: "left" | "center" | "right";
  direction?: "ltr" | "rtl";
  italic?: boolean;
  underline?: boolean;
  textTransform?: "none" | "uppercase" | "lowercase" | "capitalize";
}
```

ImageObject

```typescript
interface ImageObject extends BaseObject {
  type: "image";
  src: string; // image URL or data URL
  w: number;
  h: number;
}
```

AnswerGroupObject

```typescript
interface AnswerGroupObject extends BaseObject {
  type: "answerGroup";
  w: number;
  answers: Answer[]; // { id, text, src?, dataAnswer? }
  btnHeight: number;
  btnGap: number;
  btnPaddingTop?: number;
  btnPaddingRight?: number;
  btnPaddingBottom?: number;
  btnPaddingLeft?: number;
  btnBgColor: string;
  btnBgOpacity: number;
  btnRadius: number;
  textColor: string;
  fontSize: number;
  fontFamily?: string;
  fontWeight?: string;
  italic?: boolean;
  underline?: boolean;
  textAlign?: "left" | "center" | "right";
  direction?: "ltr" | "rtl";
  letterSpacing?: number;
  lineHeight?: number;
  textTransform?: "none" | "uppercase" | "lowercase" | "capitalize";
}
```

ShapeObject

```typescript
interface ShapeObject extends BaseObject {
  type: "shape";
  shape: "rect" | "circle";
  w: number;
  h: number;
  fill: string;
  stroke?: string;
  strokeWidth?: number;
  radius?: number; // uniform border-radius
  radiusTopLeft?: number; // per-corner overrides
  radiusTopRight?: number;
  radiusBottomRight?: number;
  radiusBottomLeft?: number;
}
```

DividerObject

```typescript
interface DividerObject extends BaseObject {
  type: "divider";
  w: number;
  thickness: number;
  color: string;
  lineStyle?: "solid" | "dashed" | "dotted";
}
```

PathObject

```
interface PathObject extends BaseObject {
  type: "path";
  d: string; // SVG path `d` attribute
  w: number; // bounding box width
  h: number; // bounding box height
  stroke?: string;
  strokeWidth?: number;
  fill?: string;
  closed?: boolean; // closed path → shows fill
}
```

Animation Types

```
interface AnimConfig {
  type: string; // preset name, e.g. "blsFadeIn"
  dur: number; // duration (ms)
  delay?: number;
  iterationCount?: number | "infinite";
  direction?: "normal" | "reverse" | "alternate" | "alternate-reverse";
  fillMode?: "none" | "forwards" | "backwards" | "both";
}

interface CustomAnim {
  name: string; // unique @keyframes rule name
  stops: KeyframeStop[];
  dur: number;
  delay?: number;
  iterationCount?: number | "infinite";
  direction?: "normal" | "reverse" | "alternate" | "alternate-reverse";
  fillMode?: "none" | "forwards" | "backwards" | "both";
  easing?: string; // overall easing curve
}

interface KeyframeStop {
  offset: number; // 0–1 (0% to 100%)
  props: Record<string, string>; // CSS prop → value
  easing?: string; // per-stop easing
}

interface LoopAnimConfig {
  type: string; // e.g. "blsFloat", "blsPulse", "blsBounce"
  dur: number;
  delay?: number;
}

interface HoverEffect {
  type: string;
} // "lift", "grow", "glow", "dim", "tilt", "brighten"
interface ClickEffect {
  type: string;
} // "pulse", "bounce", "shake", "ripple", "pop"

interface CSSFilterConfig {
  opacity?: number; // 0–100, default 100
  brightness?: number; // 0–200, default 100
  contrast?: number; // 0–200, default 100
  saturate?: number; // 0–200, default 100
  hueRotate?: number; // -180–180 deg, default 0
  blur?: number; // 0–40 px, default 0
  grayscale?: number; // 0–100, default 0
  sepia?: number; // 0–100, default 0
  invert?: number; // 0–100, default 0
}
```

Store-Level Types

```
interface ExportMeta {
  clientName: string;
  adName: string;
  adKind: string;
  countries: ExportCountry[]; // { code, languages[] }
  tracker: ExportTrackerMeta; // { enabled, endpoint }
}

interface AssetItem {
  id: string;
  name: string;
  src: string; // data URL or object URL
  addedAt: number;
}

interface CustomFontEntry {
  id: string;
  family: string;
  src: string; // data-URL for uploaded; "" for URL-loaded Google fonts
  addedAt: number;
}

interface ProjectSnapshot {
  version: number;
  quizData: QuizData;
  defaultW: number;
  defaultH: number;
  currentPreviewIndex: number;
}
```

---

6. State Management (Zustand)

Store Composition

The application uses a **single Zustand store** composed from **6 slices**, defined in `store/quizStore.ts`:

```
quizStore = create(
  frameSlice
  + objectSlice
  + selectionSlice
  + historySlice
  + clipboardSlice
  + cloudSlice
)
```

Each slice is a function `(set, get) => Partial<QuizState>` that provides a subset of state + actions.

Slice Responsibilities

Slice	File	Responsibilities
`frameSlice`	`store/slices/frameSlice.ts`	Frame CRUD, reorder, duplicate, sizing, background settings
`objectSlice`	`store/slices/objectSlice.ts`	Object add/remove/update, z-order changes, lock/hide, nudge, copy-to-all-frames
`selectionSlice`	`store/slices/selectionSlice.ts`	Single + multi-selection, active frame index, playback state, UI toggles (snap, ruler, grid, zoom, pen mode, cursor lines)
`historySlice`	`store/slices/historySlice.ts`	Undo/redo via `pastSnapshots[]` / `futureSnapshots[]` stack; `snapshot()` captures current `quizData`, `undo()` restores
`clipboardSlice`	`store/slices/clipboardSlice.ts`	Copy/paste a single `FrameObject` (new ID generated on paste)
`cloudSlice`	`store/slices/cloudSlice.ts`	Sanity project metadata: `cloudProjectId`, `cloudProjectTitle`, `cloudProjectClient`, locales, regions, `lastSavedAt`

Key Actions

```
// Frame operations
addFrame(frame)
removeFrame(index)
reorderFrame(from, to)
updateFrameField(index, Partial<Frame>)
duplicateFrame(index)
setActiveFrame(index)
commitFrameSize(index, w, h)

// Object operations
addObject(frameIndex, obj)
removeObject(frameIndex, objId)
updateObject(frameIndex, objId, updater: (obj) => obj)
duplicateObject(frameIndex, objId)
copyObjectToAllFrames(frameIndex, objId)
commitObjectPosition(frameIndex, objId, x, y)
commitObjectResize(frameIndex, objId, Partial<FrameObject>)
reorderObjectZ(frameIndex, objId, "front" | "back" | "forward" | "backward")
moveObjectToIndex(frameIndex, fromIndex, toIndex)
nudgeObject(frameIndex, objId, dx, dy)
toggleObjectVisibility(frameIndex, objId)
toggleObjectLock(frameIndex, objId)

// Selection
setSelectedObject(id | null)
toggleObjectSelection(id)        // Ctrl/Cmd+click multi-select

// History
snapshot()                        // push current state to undo stack
undo()                           // restore previous snapshot
redo()                           // restore next snapshot

// Clipboard
copyObject(frameIndex, objId)
pasteObject(frameIndex)

// Playback
setPlayback({ frameIdx, phase: "enter" | "hold" | "exit" } | null)

// Persistence
loadProject(ProjectSnapshot)
markSaved()

// Translations
setTranslation(locale, objId, text)
removeTranslationLocale(locale)
duplicateFramesAsLocale(locale)
```

Persisted Keys

The following state fields are persisted to IndexedDB under key `bls-producer-project`:

```
quizData, defaultW, defaultH, currentPreviewIndex,
snapEnabled, showRuler, showGrid, zoom,
defaultTypography,
cloudProjectId, cloudProjectTitle, cloudProjectClient,
cloudProjectLocales, cloudProjectRegions, lastSavedAt,
exportMeta, customCss, assets, customFonts
```

History System (Current)

The current undo/redo system uses **full-snapshot cloning**:

- `snapshot()` deep-clones the entire `quizData` and pushes to `pastSnapshots[]`
- `undo()` swaps current state with the top of `pastSnapshots[]`, pushes current to `futureSnapshots[]`
- `redo()` reverses the process

Limitation: Full clones are memory-heavy with large projects. The roadmap calls for migration to **operation-based undo** (JSON patches or command pattern) before the timeline phase.

---

7. Persistence & Storage

Layer 1: IndexedDB (Local)

Engine	Library	Key	Contents
IndexedDB	`idb-keyval`	`bls-producer-project`	Full `QuizData`, UI preferences, cloud metadata, assets, custom fonts

`lib/idbStorage.ts` provides the Zustand `persist` storage adapter using `idb-keyval`'s `get/set/del`.

Layer 2: localStorage

Key	Contents
`bls-panel-state-v2`	Right-panel collapse/float/position state (JSON)

Layer 3: Sanity Cloud

Endpoint	Operation	Data
`POST /api/cloud/save`	Save	Frames as JSON, image assets uploaded to Sanity CDN
`GET /api/cloud/projects`	List	All saved cloud projects
`POST /api/cloud/upload-images`	Upload	Image files → Sanity asset documents

Projects are serialised as `ProjectSnapshot` before saving.

---

8. Layout & UI Architecture

Application Shell

```
┌──────────────────────────────────────────────────┐
│  AppHeader  (toolbar — context-sensitive per selection)  │
├──────────┬──────────────────────────┬──────────────┤
│  Sidebar  │       Canvas / Board        │  RightPanel   │
│  (left)   │  (RuledCanvas → Board →     │  (5 panels)   │
│           │   FrameCards → FrameObjects) │               │
├──────────┴──────────────────────────┴──────────────┤
│  BottomPanel  (frame timeline strip)                  │
└──────────────────────────────────────────────────┘
```

AppHeader

Context-sensitive toolbar in `components/layout/AppHeader/`:

- **No selection**: Project-level controls (export, save, preview, settings)
- **Text / AnswerGroup selected**: `TextTypographyBar` appears with font family, size, weight, B/I/U, alignment, transform, letter-spacing, line-height, text colour, background colour
- Background colour picker auto-enables `bgEnabled` on the text object on colour pick

Left Sidebar

`components/layout/Sidebar/` — frame thumbnail strip, global background settings, project-level options.

RightPanel (5 Detachable Panels)

`components/layout/RightPanel/RightPanel.tsx` (513 lines)

Panel	Content	Shows When
Layers	Object list with visibility/lock toggles, dnd-kit drag-to-reorder	Always
Properties	Position (X/Y), size (W/H), rotation, type-specific fields	Object selected
Animation	Tabbed In / Out / Loop phases + Interactions (hover/click)	Object selected
Filters & FX	CSS filter sliders, blend mode, smart presets, opacity	Object selected
Frame	Frame-level animation, W/H dimensions	Always

Panel Features:

- Each panel can be **collapsed** (accordion-style)
- Each panel can be **detached** (becomes a floating portal window, draggable anywhere)
- Panel state (collapsed/floating/position) persisted to `localStorage` key `bls-panel-state-v2`
- "Dock back" button returns floating panels to the sidebar

BottomPanel

`components/layout/BottomPanel/` — horizontal strip of frame thumbnails. Supports:

- Add / duplicate / delete frames
- Drag-to-reorder
- Click to navigate to frame
- Playback preview trigger

---

9. Canvas & Rendering Engine

Component Hierarchy

```
RuledCanvas           ← ruler overlay (px markers, cursor lines)
  └── Board           ← scroll + zoom container, drop target
      └── FrameCard  ← one frame (background + objects)
          └── FrameObject (FrameObjectEl)  ← per-object wrapper
              ├── InlineTextEditor  ← TipTap (text double-click)
              ├── ImageRender       ← <img> tag
              ├── ShapeRender       ← <div> with CSS
              ├── DividerRender     ← <div> horizontal line
              ├── PathRender        ← <svg><path>
              └── AnswerGroupRender ← answer button list
```

FrameObject (`FrameObjectEl`)

The core interactive wrapper for every object on the canvas. Responsibilities:

1. **Rendering**: Delegates to type-specific renderer from `FrameObjectRenderers.tsx`
2. **Selection**: Click to select, Ctrl/Cmd+click for multi-select
3. **Drag**: Pointer-down starts drag via `DragContext`
4. **Resize**: 8 handles (4 corners + 4 edges) for resize operations
5. **Rotation**: Rotation handle for angle changes
6. **Double-click**: Activates inline editing
   - `text` → mounts `InlineTextEditor` (TipTap)
   - `answerGroup` → each answer button becomes a live `<input>`
7. **Animation playback**: During playback, reads `playback.phase` from store and applies CSS `animation` shorthand

Canvas Rendering Model

All canvas objects are rendered as **React DOM elements** with **inline** `CSSProperties`. This is an intentional design decision:

> No CSS classes on canvas objects — all styling is inline to guarantee correct export output.

The export engine reads object properties and generates matching inline styles, so the editor rendering must match exactly.

Zoom & Scroll

The `Board` component applies CSS `transform: scale(zoom)` to the frame container. Scroll position is managed by the browser's native overflow scrolling on the board wrapper.

Coordinate System

Currently uses **absolute pixel positioning** (x, y, w, h in px). Objects do not use transform matrices.

Future: Migrate to transform-matrix based coordinates for nested transforms, animation, and responsive layout (see Architecture Evolution).

---

10. Drag, Resize & Snapping

Architecture

```
hooks/useDrag.ts          — main hook: startObjectDrag(), startObjectResize()
hooks/dragHandlers.ts     — pointermove/pointerup event handlers
hooks/dragTypes.ts        — type definitions
context/DragContext.ts    — React context exposing drag handlers to FrameObject
lib/snapGuides.ts         — snap target computation
```

Drag Flow

1. `FrameObjectEl` calls `startObjectDrag(objId, e)` on pointer-down
2. `useDrag` attaches `pointermove` + `pointerup` listeners on `window`
3. On each move: computes delta, optionally runs snap checks, calls `updateObject(frameIdx, objId, updater)`
4. On pointer-up: finalises position via `commitObjectPosition()`

Resize Flow

1. Each selected object shows 8 resize handles (rendered inside `FrameObjectEl`)
2. `startObjectResize(objId, handleId, e)` maps handle ID → delta logic:
   - `"nw"`, `"ne"`, `"sw"`, `"se"` — corner resize (changes x, y, w, h)
   - `"n"`, `"s"` — vertical edge (changes y, h)
   - `"e"`, `"w"` — horizontal edge (changes x, w)
3. On pointer-up: `commitObjectResize()` persists final dimensions

Snap Guides

`lib/snapGuides.ts` computes snap targets when `snapEnabled` is true:

- **Object edges**: left, right, top, bottom of every other object
- **Object centres**: horizontal and vertical midpoints
- **Frame edges**: frame boundary lines
- Snap tolerance: typically 5px
- Visual guide lines are rendered during drag to show alignment

Multi-Select

- Ctrl/Cmd+click toggles selection via `toggleObjectSelection(id)`
- Multi-drag moves all `selectedObjectIds` in unison
- Shared property edits apply to all selected objects

---

11. Rich Text Editing (TipTap)

Integration

- **Editor**: TipTap v3 with extensions: StarterKit, Color, FontFamily, Highlight, TextAlign, TextStyle, Underline
- **Location**: `components/canvas/FrameObject/InlineTextEditor.tsx`
- **Activation**: Double-click on a `text` object mounts the TipTap editor in-place
- **Storage**: HTML output stored in `obj.richText`; takes precedence over plain `obj.text` for rendering and export

Extensions Used

Extension	Purpose
StarterKit	Bold, italic, lists, blockquote, etc.
Color	Text colour via inline styles
FontFamily	Per-span font family
Highlight	Background highlight
TextAlign	Paragraph alignment
TextStyle	Generic style attribute carrier
Underline	Text decoration

Typography Toolbar

When a text or answerGroup object is selected, `TextTypographyBar` appears in `AppHeader` with:

- Font family picker (Google Fonts + custom uploaded fonts)
- Size, weight, B/I/U toggles
- Alignment (left/centre/right)
- Text transform (uppercase/lowercase/capitalize)
- Letter spacing, line height
- Text colour picker
- Background colour picker (auto-enables `bgEnabled`)

---

12. Animation System

Three Playback Phases

Every object can have animations for each phase:

Phase	Property (preset)	Property (custom)	When
Enter	`animIn: AnimConfig`	`customAnimIn: CustomAnim`	Frame enters viewport
Exit	`animOut: AnimConfig`	`customAnimOut: CustomAnim`	Frame exits viewport
Loop	`animLoop: LoopAnimConfig`	`customAnimLoop: CustomAnim`	During hold phase (between enter and exit)

Presets

`lib/animPresets.ts` provides named preset maps:

- **Enter presets (`ANIM_IN`)**: `blsFadeIn`, `blsSlideLeft`, `blsSlideUp`, `blsZoomIn`, `blsBounceIn`, etc.
- **Exit presets (`ANIM_OUT`)**: `blsFadeOut`, `blsSlideOutLeft`, `blsZoomOut`, etc.
- **Loop presets**: `blsFloat`, `blsPulse`, `blsBounce`, `blsSwing`, etc.

Custom Keyframe Animations

Users can create custom animations with the `KeyframeEditor`:

1. Define `KeyframeStop[]` — each stop has `offset` (0–1), `props` (CSS property map), optional `easing`
2. Set duration, delay, iteration count, direction, fill mode, overall easing
3. Stored as `CustomAnim` on the object

Animation Compiler (`lib/animCompiler.ts`)

```
function compileKeyframesCSS(anim: CustomAnim): string;
```

Takes a `CustomAnim` and produces a CSS `@keyframes` rule string. Per-stop easing is injected via `animation-timing-function` inside each keyframe block.

At render time:

1. `FrameObjectEl` checks if the object has custom animations + current playback phase
2. Calls `compileKeyframesCSS()` to generate the `@keyframes` rule
3. Injects the rule into `<head>` via `useLayoutEffect` (prevents paint race)
4. Applies CSS `animation` shorthand to the element

Stagger System

Stagger creates wave-like sequential animations across objects:

- `Frame.enterStagger` / `Frame.exitStagger` — ms gap between animation order waves
- `BaseObject.animOrder` — wave index (0 = first to enter / last to exit)
- `FrameObjectEl` computes each object's delay from its `animOrder` relative to siblings
- Delay is applied as `animation-delay` on the CSS animation

Playback State

```
playback: { frameIdx: number; phase: "enter" | "hold" | "exit" } | null
```

- `null` = editing mode (no animations playing)
- `phase: "enter"` → objects play their `animIn`
- `phase: "hold"` → objects play their `animLoop`
- `phase: "exit"` → objects play their `animOut`
- Objects on non-active frames: `opacity: 0; pointer-events: none`

Interactions

Beyond phase-based animations, objects support interactive effects:

- **Hover effects**: CSS transitions triggered on `:hover` — lift, grow, glow, dim, tilt, brighten, shrink, underline
- **Click effects**: CSS animations triggered on `:active` — pulse, bounce, shake, ripple, pop

---

13. Export Engine

`lib/exportEngine.ts` (~870 lines) produces a **self-contained HTML file** with **zero external runtime dependencies** (except Google Fonts CDN).

Export Pipeline

```
QuizData
  ↓
1. Collect Google Font families + weights → <link> tags
  ↓
2. Compile all animation CSS (@keyframes + per-object animation rules)
  ↓
3. Build each frame's objects into inline-styled <div> tree
  ↓
4. Emit hover/click interaction CSS rules
  ↓
5. Embed frame transition CSS + answer stagger logic
  ↓
6. Inject embedded JS runtime for playback control
  ↓
7. Optionally inline images as base64 data URLs
  ↓
8. Assemble final HTML document
  ↓
9. Zip via fflate → download
```

Key Functions

Function	Purpose
`collectGoogleFontLinks(quizData)`	Scans all objects for font families/weights, generates `<link>` tags
`buildObjectHtml(obj, frameW, ...)`	Converts a single `FrameObject` → HTML string with inline styles + CSS rules
`compileKeyframesCSS(customAnim)`	Generates `@keyframes` CSS rule string
`emitInteractionCss(obj, cls, rules)`	Generates hover/click CSS for an object
`exportProject(quizData, exportMeta)`	Main entry point — orchestrates entire pipeline

Output Structure

```
exported.zip
  └── index.html     — self-contained (styles + JS + HTML in one file)
```

When images cannot be inlined, they appear as separate files alongside `index.html`.

Tracking Integration

If `exportMeta.tracker.enabled`, the exported HTML includes a tiny tracking script that fires events to `exportMeta.tracker.endpoint`:

- Ad load
- Frame transitions
- Answer clicks (with answer text slug)

Google Fonts Handling

- Scans all `TextObject` and `AnswerGroupObject` font families
- Parses `richText` HTML for inline `font-family` styles
- Builds Google Fonts API v2 URLs with all used weights
- Always includes 400 and 700 as baseline weights

Export Limitations (Current)

- String concatenation based (`string += "<div>"`) — not AST-based
- Runs on main thread (can block UI for large projects)
- Single output format (HTML only)
- No Web Worker offloading

Future Export Architecture

```
Scene Graph → Export Compiler → DOM AST → Serialiser → Output

DOM AST:
  ElementNode { tag, props, children[] }

Multiple serialisers:
  → HTML/CSS/JS (current)
  → React + Tailwind (.tsx)
  → Vue SFCs (.vue)
  → WordPress (PHP + CSS)
```

---

14. Sanity Cloud Integration

Sanity is used as a **cloud project/asset store** — not as a runtime CMS for the editor.

Architecture

```
Browser (editor app)
   ↓ fetch()
Next.js API Routes (/api/cloud/*)
   ↓ @sanity/client
Sanity API (cdn.sanity.io)
```

Files

Path	Purpose
`sanity/`	Sanity Studio config, schema types, client libraries
`lib/sanity/client.ts`	Sanity query client for reading assets
`lib/sanity/queries.ts`	GROQ query strings
`lib/sanity/imageUtils.ts`	Image URL builder helpers
`hooks/useSanityCloud.ts`	React hook for cloud operations (fetch/save/upload)
`app/api/cloud/save/`	POST endpoint — save project JSON + images
`app/api/cloud/projects/`	GET endpoint — list all saved projects
`app/api/cloud/upload-images/`	POST endpoint — upload image assets
`app/api/proxy-image/route.ts`	GET endpoint — CORS proxy for image URLs (needed for export inlining)

Image Proxy

The export engine needs to inline images as base64. Cross-origin images cannot be read from canvas due to CORS. The `/api/proxy-image` route fetches images server-side and returns them without CORS restrictions.

---

15. Internationalisation (i18n)

Translation Model

```
QuizData.translations: Record<string, Record<string, string>>
// locale → objectId → translated text
```

Workflow

1. User creates the primary-language frames
2. `duplicateFramesAsLocale(locale)` clones all frames with the target locale tag
3. Per-object translations are stored in the `translations` map
4. Export engine can produce locale-specific outputs

Frame Locale Tagging

```
Frame.locale?: string  // e.g. "ar", "en", "fr"
```

Duplicated frames carry the locale identifier, allowing the export engine and preview system to filter by language.

---

16. Coding Conventions & CI

TypeScript

- **Strict mode**: `strict: true` in `tsconfig.json`
- **No emit**: `noEmit: true` — TypeScript is used only for type checking
- **Path alias**: `@src/*` maps to the project root

ESLint

- ESLint 9 flat config (`eslint.config.mjs`)
- `--max-warnings=0` in CI — **zero warnings allowed**
- `lint-staged` runs `eslint --fix --max-warnings=0` on staged files at commit time

Commits

- **Conventional Commits** enforced via `@commitlint/cli` + `@commitlint/config-conventional`
- `husky` runs commitlint + lint-staged on `pre-commit`

Canvas Styling Rule

> **No global CSS classes for canvas objects.** All canvas styling is inline `CSSProperties` to guarantee correct export output.

This is critical: the export engine reads object properties and generates inline styles. If canvas rendering used CSS classes, the export output would not match the editor preview.

Rich Text Precedence

> When `obj.richText` is present, it takes precedence over `obj.text` for both rendering and export.

---

17. Feature Priority List

Priority 1 — Must-Have DSP-Safe Core

Vector/Pen Tool:

- Each language in its own tab
- Curve pen tool for freeform vector shapes
- Shift key constrains to straight lines
- Connect any vector lines/shapes together
- Created shapes have: bg colour, stroke colour, stroke width, stroke type controls
- Select edges of vector shapes to re-edit them (SVG editor)

Effects Engine:

- Liquid distortion layer (CSS `backdrop-filter` + mask)
- Particle emitters (CSS keyframe particles)
- 3D card tilt / gyroscope effect (CSS `perspective` + `rotateX/Y`)
- Dynamic shadows reacting to cursor (CSS variables + `drop-shadow`)
- Text scramble / decoding animation (CSS `content` + keyframes)
- SVG path drawing animations (`stroke-dasharray` + `stroke-dashoffset`)
- Shape morphing (CSS `clip-path` + transition)
- Tap / hover interactions (magnetic hover, tap-to-catch)

Priority 2 — High Impact Templates

- Scratch card reveal (CSS mask + hover/tap)
- Slot machine (CSS keyframe reels)
- Spin the wheel (CSS keyframe rotation)
- Swipe comparison (before/after slider)
- Memory card game (CSS flip animation)
- Puzzle assemble game (CSS draggable + snap via CSS grid)
- Product unboxing animation (CSS 3D transforms + layering)
- Story/reels style vertical ad (CSS scroll snap + parallax)
- Candy crush / match game (CSS grid + animations)
- Maze game (CSS grid + hover/tap path highlighting)
- 360 panorama (CSS perspective + rotate via keyframes)

Priority 3 — Advanced Features & Optimisation

- Timeline animation editor (CSS variables + keyframes)
- Drag physics system (CSS snap points + transitions)
- Responsive auto-layout (CSS grid + flexbox + `clamp()`)
- Smart asset compression (optimised sprites + CSS only)
- Parallax layers (CSS perspective + transform + scroll-linked variables)
- Scroll triggered animations (CSS scroll-linked keyframes)
- Dynamic theme switcher (CSS variables for dark/light mode)
- AI auto-animate still images (pre-rendered sprite sheets)
- Product builder (customisable CSS layers)

Priority 4 — Power Features / Extras

- Shader-like effects (CSS filter stack + backdrop-filter)
- Procedural geometry generator (CSS clip-path + mask)
- Interactive data visualisations (CSS counters + pseudo-elements)
- Real-time multiplayer mini games (CSS only logic + fallback images)
- Ad performance heatmap replay (pre-rendered overlays)
- Smart asset swap (CSS variables + preloaded sprite sets)
- Liquid morphing backgrounds (CSS gradients + clip-path transitions)
- AI auto-generate ad layouts (pre-generated CSS classes)

---

18. Architecture Evolution Plan

Current Architecture (v1.x)

```
React UI → Zustand Store → Frame/Object Model → Export Engine
```

Target Architecture (v3.x+)

```
Client Editor
│
├── UI Layer
│     React + Mantine Panels + Tools
│
├── Editor Engine
│     Command System (do/undo)
│     Object Model (scene graph)
│     Interaction Engine
│     Timeline Engine
│
├── Rendering Engine
│     Hybrid SVG + Canvas2D
│     DOM overlay for handles/UI
│
├── Data Layer
│     projectStore (nodes, assets, pages, tokens)
│     uiStore (selection, tools, panels)
│     historyStore (command stack)
│     runtimeStore (playback, preview)
│
└── Worker Layer
      exportWorker
      animationCompileWorker
      vectorBooleanWorker
      imageProcessingWorker
```

Critical Architectural Migrations

1. Command Pattern (Undo System)

Current: Full-snapshot cloning (`pastSnapshots[]`)
Target: Command-based undo with `do()` / `undo()` interface

```
interface Command {
  do(): void;
  undo(): void;
}

// Example commands:
MoveObjectCommand;
ResizeObjectCommand;
CreateObjectCommand;
DeleteObjectCommand;
UpdatePropertyCommand;
AddEffectCommand;
AddKeyframeCommand;
```

Benefits:

- Memory-efficient (stores operations, not full state clones)
- Multiplayer-ready (commands can be serialised and replayed)
- Macro recording (record command sequences)
- Infinite undo without memory explosion

When: Must be implemented before Phase 3 (Timeline).

2. Transform Matrix System

Current: Simple `x, y, w, h, rotation` fields
Target: Transform matrix `[a, b, c, d, e, f]` per object

```
matrix encodes: translate, rotate, scale, skew
```

Benefits:

- Nested transforms (groups, components)
- Timeline animation of compound transforms
- Prefab instancing
- Responsive layout

When: Must be decided before Phase 1 (Vector Engine).

3. Scene Graph Object Model

Current: Flat `FrameObject[]` array per frame
Target: Recursive scene graph

```
Node
 ├── id
 ├── type
 ├── transform
 ├── style
 ├── effects[]
 ├── animations[]
 ├── interactions[]
 └── children[]

Scene → Frame → Node → Node → Node …
```

When: Gradual migration starting Phase 1.

4. Hybrid Renderer

Current: Pure React DOM with inline styles
Target: Hybrid DOM + Canvas2D

```
EditorCanvas
 ├── Layer: Interaction overlay (DOM) — handles, selection boxes
 ├── Layer: Vector canvas (Canvas2D) — vector editing, heavy scenes
 └── Layer: UI handles (DOM) — resize handles, rulers
```

Why: DOM dies above ~300 nodes with transforms. Canvas handles 10k+ shapes easily.

When: Must be decided before Phase 1 (Vector Engine).

5. Store Split

Current: Single Zustand store with 6 slices
Target: Multiple purpose-specific stores

```
projectStore  — nodes, assets, pages, tokens
uiStore       — selectedIds, tool, panels, zoom
historyStore  — command stack
runtimeStore  — playback state, preview
```

Rule: Never mix UI state with project data.

6. Effect System

Current: Direct CSS filter properties on objects
Target: Composable effect definitions

```
interface Effect {
  type: "3dTilt" | "liquidDistortion" | "particleEmitter" | ...;
  trigger: "always" | "hover" | "click" | "scroll" | "viewport-enter";
  params: Record<string, unknown>;
}
```

Pipeline: `Effect Definition → Effect Compiler → Runtime Output (CSS / JS / SVG filters)`

7. Asset System

Current: Images stored as data URLs in objects, custom fonts as data URLs
Target: First-class asset model

```
interface Asset {
  id: string;
  type: "image" | "video" | "audio" | "font" | "svg" | "lottie";
  src: string;
  metadata: Record<string, unknown>;
}
```

Storage: IndexedDB (local) → Cloud CDN (future).

8. Export Engine Evolution

Current: String concatenation
Target: AST-based DOM tree builder → serialise to multiple formats

```
Scene Graph → Export Compiler → DOM AST (ElementNode) → Serialiser → HTML / React / Vue / WordPress
```

Plus: Web Worker offloading for non-blocking export.

---

19. Roadmap — Phase-by-Phase

Milestone Overview

Milestone	Version	Scope	Est. Effort
Phase 0	v1.5	UX debt & polish	~2 weeks
Phase 1	v2.0	Vector engine + pen tool	~8 weeks
Phase 2	v2.5	Effects engine + interactions	~10 weeks
Phase 3	v3.0	Timeline & animation studio	~12 weeks
Phase 4	v3.5	Game templates & prefabs	~8 weeks
Phase 5	v4.0	Responsive layout engine	~12 weeks
Phase 6	v5.0	Web design suite	~20 weeks

Phase 0 — UX Debt & Polish (v1.5, ~2 weeks)

#	Task	Scope
0.1	Implement remaining 4-sided border-radius picker	ShapeObjectFields
0.2	Add “Paste in place” (identical x/y to source)	clipboardSlice
0.3	Multi-object alignment toolbar (align left/right/top/bottom/distribute)	toolbar + lib/align.ts
0.4	Object grouping (temporary “lock together” for multi-drag)	selectionSlice
0.5	Preview overlay (full-screen animated playback inside editor)	AdPreviewModal
0.6	Keyboard shortcut improvements & discoverability	KeyboardShortcutsModal
0.7	Panel state auto-restore on window resize	RightPanel
0.8	Performance audit: reduce unnecessary re-renders in FrameObjectEl	React.memo / selectors

---

Phase 1 — Vector Engine (v2.0, ~8 weeks)

New Object Type: `VectorPathObject`

```typescript
interface VectorPathObject extends BaseObject {
  type: "vector";
  nodes: VectorNode[]; // control points
  segments: VectorSegment[]; // connecting segments
  closed: boolean;
  fill?: string;
  fillRule?: "evenodd" | "nonzero";
  stroke?: string;
  strokeWidth?: number;
  strokeLinecap?: "butt" | "round" | "square";
  strokeLinejoin?: "miter" | "round" | "bevel";
  strokeDasharray?: string;
  w: number;
  h: number;
}
```

Pen Tool Behaviour

Action	Result
Click	Add anchor (straight segment)
Click + drag	Add anchor with control handles (curve)
Shift + click	Constrain to 45° increments
Hover over start node	Close path
Escape	End open path
Double-click	End open path and deselect

Node Editing

Action	Result
Click node	Select node
Drag node	Move node + neighbours update
Drag control handle	Adjust bézier curvature
Alt + drag handle	Break tangent symmetry
Delete key	Remove node (path reshapes)
Double-click segment	Insert node at point

SVG Property Panel

Stroke, fill, opacity, stroke-dasharray, linecap, linejoin controls.

Open Questions

#	Question	Options
Q1	Coordinate system — transform matrix or keep hybrid?	Matrix (future-proof) · Hybrid (simpler now)
Q2	Rendering — inline SVG in React DOM, or Canvas 2D?	SVG (consistent with export) · Canvas (perf)
Q3	Export — vectors as inline `<svg>` in HTML output?	Yes · No (rasterise)
Q4	Fill gradients — shared definitions or per-object?	Shared (Figma-style) · Per-object
Q5	Existing `PathObject` — deprecate in favour of `VectorPathObject`?	Deprecate · Keep both
Q29	Hybrid renderer cutover point	Before Phase 1 · During Phase 1

Phase 2 — Interactive Components & Effects Engine (v2.5, ~10 weeks)

Effect System

New composable `effects: EffectConfig[]` array on every `BaseObject`:

Effect	CSS Technique	Export Method
Liquid distortion	`backdrop-filter: url(#turbulence)` + SVG `<feTurbulence>`	Inline SVG filter
Particle emitter	N pseudo-elements with staggered `@keyframes`	Generated CSS
3D card tilt	`perspective() rotateX(var(--tiltX)) rotateY(var(--tiltY))`	JS mousemove + CSS vars
Dynamic shadows	`drop-shadow(var(--sx) var(--sy) var(--sb) color)`	CSS vars + JS
Text scramble	`content` cycling via `@keyframes steps()`	Pure CSS
SVG path draw	`stroke-dasharray` + `stroke-dashoffset` animation	CSS animation
Shape morph	`clip-path: polygon()` transition	CSS transition + JS trigger
Magnetic hover	`transform` on pointer proximity	JS pointermove
Parallax layer	`perspective` + `translateZ`	CSS + scroll-linked

Each effect has:

- `trigger`: `always | hover | click | scroll | viewport-enter`
- `params`: type-specific configuration (intensity, colour, speed, etc.)

Interaction System Overhaul

Replace current `HoverEffect` / `ClickEffect` with a full interaction model:

Trigger	Actions
`hover`	Play animation · Toggle visibility · Change state variable
`click`	Navigate to frame · Open URL · Play sound · Trigger effect
`long-press`	Same as click actions
`swipe`	Navigate forward/back · Dismiss element
`drag`	Move element · Snap to target

Actions can target `self` or any other object by ID.

Open Questions

#	Question
Q6	Should export JS be inlined or bundled as a separate `bls-runtime.js`?
Q7	Can multiple effects stack on one object simultaneously?
Q8	Add audio assets to the asset model now?
Q9	How far should the state-variable / reactive binding system go?
Q10	Expose a “performance mode” toggle to disable effects during editing?

Phase 3 — Timeline & Animation Studio (v3.0, ~12 weeks)

Timeline Data Model

Replace simple `AnimConfig` with property-level keyframe tracks:

```
interface TimelineTrack {
  objectId: string;
  property: "x" | "y" | "opacity" | "rotation" | "scale" | ...;
  keyframes: TimelineKeyframe[];
}

interface TimelineKeyframe {
  time: number;                  // ms from frame start
  value: number | string;
  easing: EasingFunction;
}
```

Runtime: Compile to **Web Animations API** (not CSS) for timeline control, scrubbing, reversing, pause/resume, programmatic control.

Timeline UI

```
┌──────────────────────────────────────────────────┐
│ ▶ ⏸ ■ │ 0:00.000  ←──playhead──→      0:05.000   │
├──────────┼───────────────────────────────────────┤
│ Object 1 │ ▓▓▓    ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ │
│   ├ x    │ ◆──────────◆                          │
│   ├ y    │ ◆──◆──────────────◆                   │
│   └ opa  │ ◆──────────────────────────────────◆  │
│ Object 2 │ ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ │
│   └ x    │      ◆──────────◆                     │
├──────────┼───────────────────────────────────────┤
│ Graph Editor (toggle)  ╱╲────╱╲                  │
└──────────────────────────────────────────────────┘
```

Features:

- Drag keyframe diamonds to re-time
- Right-click keyframe → change easing curve
- Graph editor for visual bézier easing editing
- Scrub playhead for real-time canvas preview
- Onion-skinning toggle (ghost previous/next keyframe)

Migration Path

Old Model	New Model	Compatibility
`animIn` preset	Auto-generates timeline tracks	Presets still usable as shortcuts
`animOut` preset	Same	Same
`animLoop` preset	Infinite-repeat track	Same
`customAnim`	Direct keyframe import	1:1 mapping

User can "Convert to timeline" to unlock full keyframe editing.

Open Questions

#	Question
Q11	Frames as independent "scenes" with own timelines, or one global timeline with scene markers?
Q12	Include a bézier curve editor for easing (like After Effects)?
Q13	Real-time canvas update on playhead scrub?
Q14	Use Web Animations API in export for more control?
Q15	Auto-migrate old projects to timeline format on load?

Phase 4 — Game Templates & Prefabs (v3.5, ~8 weeks)

Prefab System

```typescript
interface Prefab {
  id: string;
  name: string;
  category: string;
  thumbnail: string;
  objects: FrameObject[];
  interactions: InteractionConfig[];
  requiredVariables: VariableDefinition[];
  customCss?: string;
  customJs?: string; // sandboxed runtime
}
```

Game Prefab Catalogue

Template	Technique	Complexity
Scratch card reveal	CSS `mask-image` + pointer tracking	Medium
Slot machine	CSS `@keyframes` on 3 reel strips	Medium
Spin the wheel	CSS `rotate` + JS random stop angle	Medium
Before / after slider	CSS `clip-path` + drag handler	Low
Memory card game	CSS `rotateY(180deg)` flip + match logic	High
Puzzle assemble	CSS grid + drag-snap + completion check	High
Product unboxing	Sequenced 3D transforms	Medium
Story / reels ad	CSS scroll-snap + parallax	Medium
Swipe comparison	Range input + `clip-path`	Low
Maze game	CSS grid + hover/tap path highlighting	High
360 panorama	CSS perspective + rotate via keyframes	Medium

Prefab Distribution

```
Local (bundled) → Sanity CMS (curated) ← Community (future)
```

Open Questions

#	Question
Q16	Should prefab JS run in an iframe sandbox or inline with restricted API?
Q17	Should game results feed into the existing tracker system?
Q18	Should prefabs expose configurable difficulty parameters?
Q19	Can users “break open” a prefab and edit internals?

Phase 5 — Responsive Layout Engine (v4.0, ~12 weeks)

Layout Modes

Add `layoutMode` to frames:

Mode	Behaviour
`absolute`	Current behaviour — free-positioned objects
`flex`	Figma-style auto-layout (direction, gap, padding, alignment)
`grid`	CSS grid with row/column definitions
`auto`	AI-suggested layout based on object positions

Breakpoints

```
interface Breakpoint {
  maxWidth: number;
  overrides: Partial<Frame>; // layout/sizing changes at this width
}
```

Objects can have per-breakpoint overrides for position, size, visibility, and font size.

Device Preview Bar

Category	Presets
Mobile	375×667, 390×844, 414×896
Tablet	768×1024, 834×1194
Desktop	1280×720, 1440×900, 1920×1080

Open Questions

#	Question
Q20	Is responsive layout needed for ads too, or only web design mode?
Q21	CSS media queries, container queries, or JS resize observer?
Q22	Can a frame mix absolute and flex/grid children?
Q23	Should responsive projects export as multi-file static sites?

Phase 6 — Web Design Suite (v5.0, ~20 weeks)

Multi-Page Projects

```
interface Project {
  type: "ad" | "website" | "prototype";
  pages: Page[];
  globalStyles: StyleDefinition[];
  designTokens: TokenSet;
  assets: AssetLibrary;
}

interface Page {
  id: string;
  name: string;
  slug: string;
  frame: Frame; // reuses existing frame as page content
  meta: PageMeta;
}
```

Component System

Reusable component definitions with instances and overrides:

- **Definition**: name, props, template object tree, variants
- **Instance**: references definition by ID, stores per-instance overrides
- **Propagation**: editing a definition updates all instances

Design Tokens

```
interface TokenSet {
  colors:       { primary: string; secondary: string; ... };
  typography:   { heading: { family, size, weight, lineHeight }; ... };
  spacing:      { sm: 4; md: 8; lg: 16; ... };
  borderRadius: { sm: 4; md: 8; full: 9999 };
  shadows:      { sm: string; md: string; ... };
  breakpoints:  { mobile: 640; tablet: 1024; desktop: 1440 };
}
```

Objects reference tokens by name (`$color.primary`), enabling theme switching (dark/light mode, brand variants).

Code Export Targets

Format	Use Case
Static HTML / CSS / JS	Current ad export, expanded for multi-page sites
React + Tailwind	`.tsx` components
Vue SFCs	`.vue` single-file components
WordPress theme	PHP templates + CSS
Figma import / export	Design file interop

Open Questions

#	Question
Q24	Should “ad” and “website” be separate project types from creation?
Q25	Should components be global (cross-project) or per-project?
Q26	Is real-time multi-user editing on the roadmap? (CRDT sync via Yjs)
Q27	Should the suite include a “publish to CDN” flow?
Q28	Should third-party plugins be supported? (Plugin API + sandbox)

---

20. Cross-Cutting Concerns

Performance Scaling Targets

Metric	Current	Target
Objects per frame	~50 comfortable	500+ (DOM) / 1000+ (Canvas)
Vector nodes	~200	10,000+
Animations per frame	~20	500
Frames per project	~20	100+ pages
Assets per project	~50	2,000+

Worker Architecture

Heavy operations must leave the main thread:

Worker	Purpose
`exportWorker`	HTML generation + zip compression
`animationCompileWorker`	Keyframe CSS compilation for complex timelines
`vectorBooleanWorker`	Path boolean operations (union, subtract, intersect)
`imageProcessingWorker`	Resize, compress, format conversion

Testing Strategy

Level	Scope	Priority
Unit	Export engine, alignment math, animation compiler	High
Unit	Store slices (frame/object CRUD, undo/redo)	High
Snapshot	Export HTML output regression tests	Medium
Component	Sidebar panels, toolbar interactions	Medium
E2E	Full create → edit → export flow	Low (later)

Plugin System (Future v6)

```
registerEffect();
registerPrefab();
registerExporter();
registerTool();
```

Sandbox: iframe-based worker isolation for third-party code.

---

21. Open Architecture Decisions

Decision Status Legend

- 🔴 **Blocking** — must answer before starting that phase
- 🟡 **Important** — should answer before mid-point of that phase
- 🟢 **Non-blocking** — can decide during implementation

Full Decision Register

#	Decision	Status	Phase
Q1	Coordinate system: transform-matrix vs hybrid	🔴	Phase 1
Q2	Rendering: SVG DOM vs Canvas 2D for design surface	🔴	Phase 1
Q3	Vector export format: inline SVG or rasterise?	🔴	Phase 1
Q4	Gradient definitions: shared (Figma-style) or per-object?	🟡	Phase 1
Q5	PathObject: deprecate in favour of VectorPathObject?	🟡	Phase 1
Q6	Export JS: inline or separate `bls-runtime.js`?	🟡	Phase 2
Q7	Effect stacking: can multiple effects stack on one object?	🟡	Phase 2
Q8	Audio asset model: add now or defer?	🟡	Phase 2
Q9	State variable / reactive binding: how deep?	🟡	Phase 2
Q10	Performance mode toggle during editing?	🟢	Phase 2
Q11	Frames as scenes with own timelines, or one global timeline?	🔴	Phase 3
Q12	Bézier curve editor for easing?	🟡	Phase 3
Q13	Real-time canvas update on playhead scrub?	🟡	Phase 3
Q14	Web Animations API in export?	🟡	Phase 3
Q15	Auto-migrate old projects to timeline format?	🟡	Phase 3
Q16	Prefab JS sandbox: iframe or restricted API?	🟡	Phase 4
Q17	Game results → tracker integration?	🟡	Phase 4
Q18	Prefab configurable difficulty parameters?	🟡	Phase 4
Q19	Users can “break open” prefab internals?	🟡	Phase 4
Q20	Responsive layout: ads + web, or web only?	🟡	Phase 5
Q21	Responsive: media queries, container queries, or resize observer?	🟡	Phase 5
Q22	Mix absolute + flex/grid children in one frame?	🟡	Phase 5
Q23	Responsive export: single HTML or multi-file static site?	🟡	Phase 5
Q24	Separate “ad” vs “website” project types?	🟡	Phase 6
Q25	Components: global (cross-project) or per-project?	🟡	Phase 6
Q26	Real-time multiplayer editing (CRDT / Yjs)?	🟡	Phase 6
Q27	“Publish to CDN” flow?	🟡	Phase 6
Q28	Third-party plugin API + sandbox?	🟡	Phase 6

#	Decision	Status	Phase
Q29	Hybrid renderer cutover point	🔴	Phase 1
Q30	Undo system migration to command pattern	🟡	Phase 3

The Five Critical Locks

These architectural decisions **cannot wait** — they must be locked before Phase 2 to prevent cascading rework:

1. **Transform matrix system** — affects how every object stores position/rotation/scale
2. **Hybrid renderer** — determines canvas component architecture
3. **Command-based undo** — affects every state mutation in the app
4. **Scene graph object model** — determines data structure for all future features
5. **Worker export pipeline** — affects export engine rewrite scope

> ⚠️ The **danger zone starts at Phase 2** (v2.5) when effects, interactions, timeline, and prefabs all collide. Without strong architecture locked by then, the codebase becomes unmaintainable within ~6 months.

End of documentation.